

Лабораторная работа №6

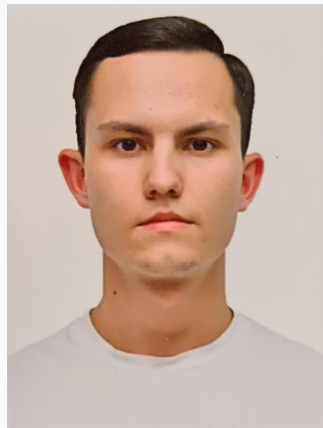
Решение моделей в непрерывном и дискретном времени

Дурдалыев Максат

2025-11-22

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

- Дурдалыев Максат
- Студент НКНбд-01-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132205337@pfur.ru
- <https://github.com/mdurdalyyev>



Цель работы

Основной целью работы является освоение специализированных пакетов для решения задач в непрерывном и дискретном времени.

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы.

3 Решение обыкновенных дифференциальных уравнений

Вспомним, что обыкновенное дифференциальное уравнение (ОДУ) описывает изменение некоторой переменной u .

Для решения обыкновенных дифференциальных уравнений (ОДУ) в Julia можно использовать пакет `diffrentialEquations.jl`.

4 Модель экспоненциального роста

```
▶ [ ] # подключаем необходимые пакеты:
import Pkg
Pkg.add("DifferentialEquations")

[2] ✓ 4.9s using DifferentialEquations

[4] ✓ 0.3s # задаём описание модели с начальными условиями:
a = 0.98
f(u,p,t) = a*u
u0 = 1.0
# задаём интервал времени:
tspan = (0.0,1.0)

... (0.0, 1.0)

[5] ✓ 3.4s # решение:
prob = ODEProblem(f,u0,tspan)
sol = solve(prob)

... retcode: Success
Interpolation: 3rd order Hermite
t: 5-element Vector{Float64}:
 0.0
 0.100424944449239292
 0.3521860297865888
 0.6934436122197829
 1.0
u: 5-element Vector{Float64}:
 1.0
```

5 Модель экспоненциального роста

```
# подключаем необходимые пакеты:  
Pkg.add("Plots")  
using Plots
```

[6]

✓ 8.7s

Julia

```
... Resolving package versions...
```

```
Project No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Proj`
```

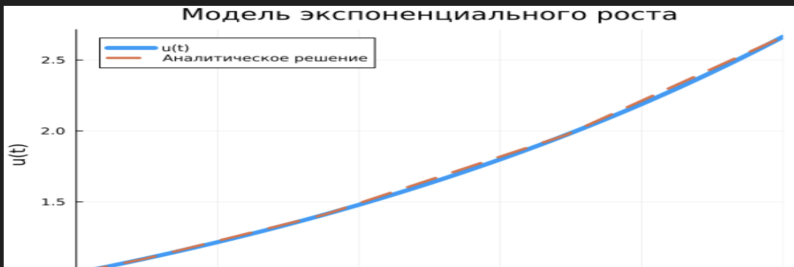
```
Manifest No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Mani`
```

```
# строим графики:  
plot(sol, linewidth=5, title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label=:none)  
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, label="Аналитическое решение")
```

[7]

✓ 5.8s

Julia

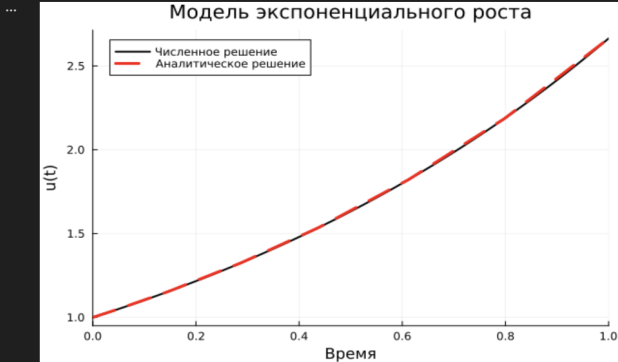


6 Модель экспоненциального роста

```
# задаём точность решения:
sol = solve(prob, abstol=1e-8, reltol=1e-8)
println(sol)
# строим график:
plot(sol, lw=2, color="black", title="Модель экспоненциального роста", xaxis="Время", yaxis="u(t)", label="Численное решение")
plot!(sol.t, t->1.0*exp(a*t), lw=3, ls=:dash, color="red", label="Аналитическое решение")
```

[8] ✓ 4.1s

... ODESolution{Float64, 1, Vector{Float64}, Nothing, Nothing, Vector{Float64}, Vector{Vector{Float64}}, Nothing, ODEProblem{Float64, Tuple{



7 Система Лоренца

Динамической системой Лоренца является нелинейная автономная система обыкновенных дифференциальных уравнений третьего порядка.

Система получена из системы уравнений Навье–Стокса и описывает движение воздушных потоков в плоском слое жидкости постоянной толщины при разложении скорости течения и температуры в двойные ряды Фурье с последующем усечением до первых-вторых гармоник.

Решение системы неустойчиво на аттракторе, что не позволяет применять классические численные методы на больших отрезках времени, требуется использовать высокоточные вычисления.

8 Система Лоренца

```
# задаём описание модели:
function lorenz!(du,u,p,t)
    β,ρ,β = p
    du[1] = β*(u[2]-u[1])
    du[2] = u[1]*(ρ-u[3]) - u[2]
    du[3] = u[1]*u[2] - β*u[3]
end
# задаём начальное условие:
u0 = [1.0,0.0,0.0]
# задаём значения параметров:
p = (10,28,8/3)
# задаём интервал времени:
tspan = (0.0,100.0)
# решение:
prob = ODEProblem(lorenz!,u0,tspan,p)
sol = solve(prob)
# подключаем необходимые пакеты:
Pkg.add("Plots")
using Plots
# строим график:
plot(sol, vars=(1,2,3), lw=2, title="Аттрактор Лоренца", xaxis="x", yaxis="y", zaxis="z", legend=false)
44] ✓ 14.1s

Resolving package versions...
Project No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Project.toml`
Manifest No packages added to or removed from `C:\Users\durda\.julia\environments\v1.12\Manifest.toml`
```

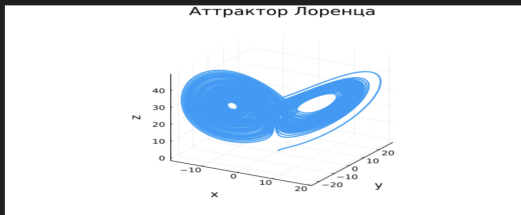
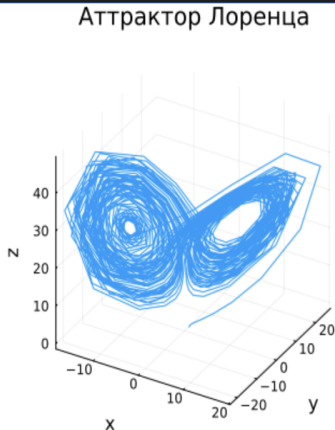


Рисунок 4: Аттрактор Лоренца

9 Система Лоренца

```
▶ ▾  
● # отключаем интерполяцию:  
plot(sol,vars=(1,2,3),denseplot=false, lw=1, title="Аттрактор Лоренца", xaxis="x",yaxis="y", zaxis="z",legend=false)  
[12] ✓ 0.1s
```



10 Модель Лотки–Вольтерры



Рисунок 6: Модель Лотки–Вольтерры: динамика изменения численности популяций

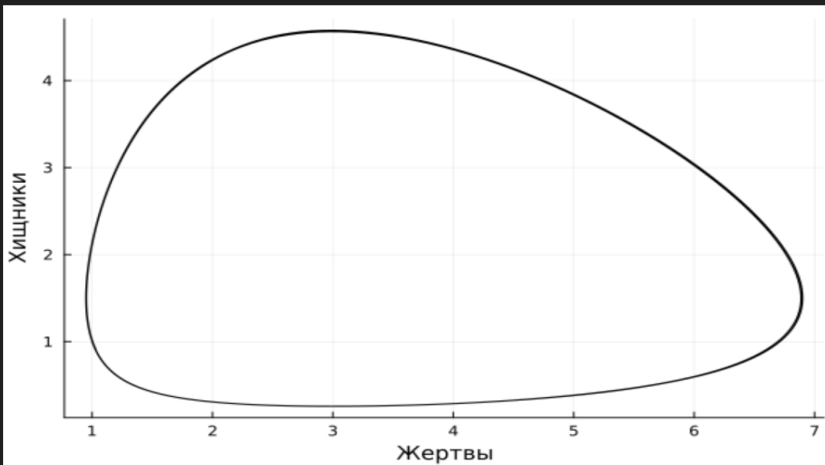
11 Модель Лотки–Вольтерры

```
# фазовый портрет:
```

```
plot(sol,vars=(1,2), color="black", xaxis="Жертвы",yaxis="Хищники", legend=false)
```

[16] ✓ 0.2s

...



12 Самостоятельное выполнение

Задания

1. Реализовать и проанализировать модель роста численности изолированной популяции (модель Мальтуса). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

🔗 Создать + Code + Markdown

```
f(u, p, t) = a * u
b = 0.5
c = 0.1
a = b - c
u0 = 1.0
tspan = (0.0, 10.0)
prob = ODEProblem(f, u0, tspan)
sol = solve(prob)
plot(sol, title="Модель Мальтуса", label="Численное решение", color="pink", linewidth = 3)
```

17)

✓ 1.8s

Julia

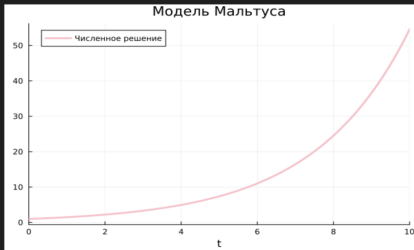


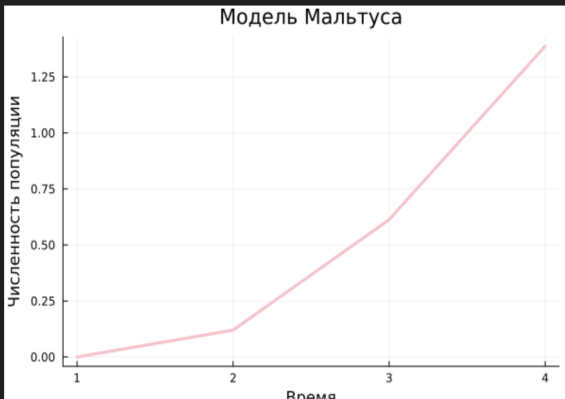
Рисунок 8: Решение задания №1

13 Самостоятельное выполнение

```
anim = @animate for i in 1:length(sol.t)
    plot(sol.t[1:i], linewidth=3, title="Модель Мальтуса", xlabel="Время", ylabel="Численность популяции", legend=false, color="pink")
end
gif(anim, "malthus_population_growth.gif", fps=15)
```

[18] ✓ 1.1s

... Info: Saved animation to c:\Users\durda\OneDrive\Desktop\malthus_population_growth.gif
@ Plots <C:\Users\durda\julia\packages\Plots\8ZnR3\src\animation.jl:156>



14 Самостоятельное выполнение

2. Реализовать и проанализировать логистическую модель роста популяции. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
f(u, p, t) = r * u * (1 - u/k)
r = 0.5
k = 100
u0 = 1.0
tspan = (0.0, 20.0)
prob = ODEProblem(f, u0, tspan)
sol = solve(prob)
plot(sol, title="Логистическая модель роста популяции", label="Численность популяции", color="green", linewidth = 3)
```

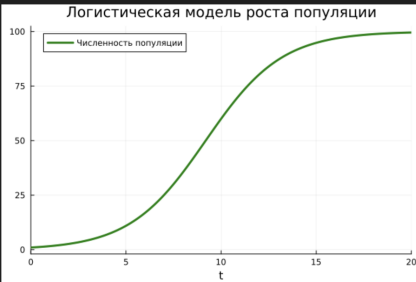


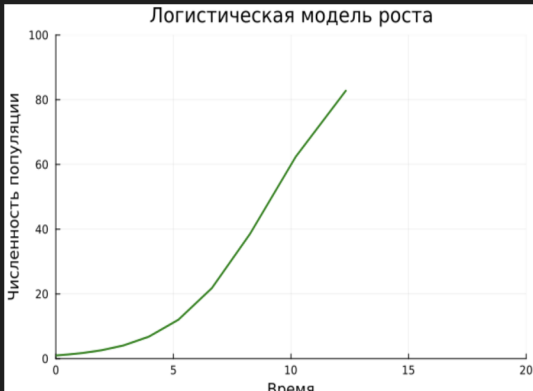
Рисунок 10: Решение задания №2

15 Самостоятельное выполнение

```
anim = @animate for t in 0:0.5:20
    plot(sol.t[sol.t .<= t], sol.u[sol.t .<= t], label="Численное решение", xlim=(0, 20), ylim=(0, k), lw=2, title="Логистическая модель роста", x
end
gif(anim, "logistic_growth.gif", fps=10)
```

[25] ✓ 1.2s

Info: Saved animation to c:\Users\durda\OneDrive\Desktop\logistic_growth.gif
@ Plots <C:\Users\durda\.julia\packages\Plots\8ZnR3\src\animation.jl:156>



3. Реализовать и проанализировать модель эпидемии Кермака-Маккендрика (SIR-модель). Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией):

```
function sir(du, u, p, t)
    s, i, r = u
    β, γ = p
    du[1] = -β * s * i # ds/dt
    du[2] = β * s * i - γ * i # di/dt
    du[3] = γ * i # dr/dt
end
```

17 Самостоятельное выполнение

```
# Начальные данные
u0 = [0.95, 0.05, 0.0] # 95% восприимчивых, 5% инфицированных, 0% выздоровевших

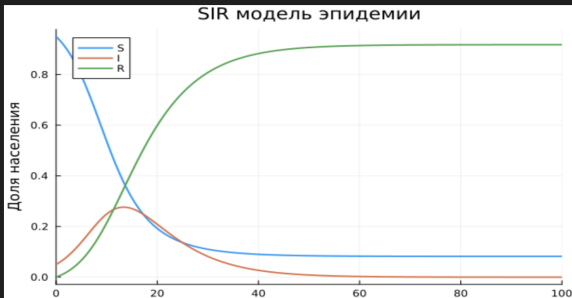
# Параметры
β = 0.4 # Коэффициент передачи инфекции
γ = 0.15 # Коэффициент выздоровления

p = (β, γ)
tspan = (0.0, 100.0)

prob_sir = ODEProblem(sir, u0, tspan, p)
sol_sir = solve(prob_sir, Tsit5())

plot(sol_sir, title="SIR модель эпидемии", label=["S" "I" "R"], linewidth=2)
xlabel!("Время (дни)")
ylabel!("Доля населения")
```

✓ 1.7s



18 Самостоятельное выполнение



Рисунок 14: Решение задания №3

19 Самостоятельное выполнение

4. Как расширение модели SIR (Susceptible-Infected-Removed) по результатам эпидемии испанки была предложена модель SEIR (Susceptible-Exposed-Infected-Removed). Исследуйте, сравните с SIR.

```
function seir(du, u, p, t)
    s, e, i, r = u
    β, σ, γ = p
    N = s + e + i + r # Общая численность населения

    du[1] = -β * s * i / N      # ds/dt: Восприимчивые
    du[2] = β * s * i / N - σ * e # de/dt: Экспонированные
    du[3] = σ * e - γ * i       # di/dt: Инфицированные
    du[4] = γ * i               # dr/dt: Выздоровевшие
end
```

[29] ✓ 0.0s

Jul

20 Самостоятельное выполнение

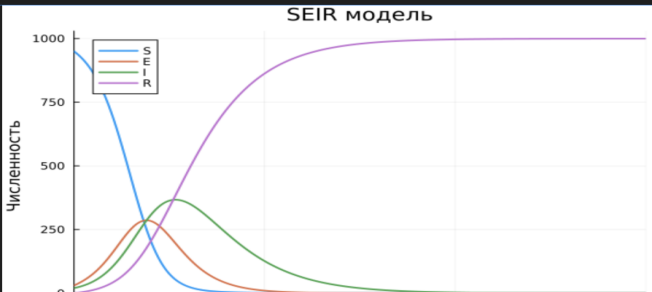
```
# начальные данные
u0 = [950.0, 30.0, 20.0, 0.0] # S=950, E=30, I=20, R=0 (всего 1000)

# параметры
p = [0.6, 0.15, 0.08] #  $\beta=0.6$ ,  $\sigma=0.15$ ,  $\gamma=0.08$ 

tspan = (0.0, 150.0) # время
prob_seir = ODEProblem(seir, u0, tspan, p)
sol_seir = solve(prob_seir, Tsit5())

plot(sol_seir, title="SEIR модель", label=["S" "E" "I" "R"], linewidth=2)
xlabel!("Время (дни)")
ylabel!("Численность")
```

[30] ✓ 1.3s



21 Самостоятельное выполнение

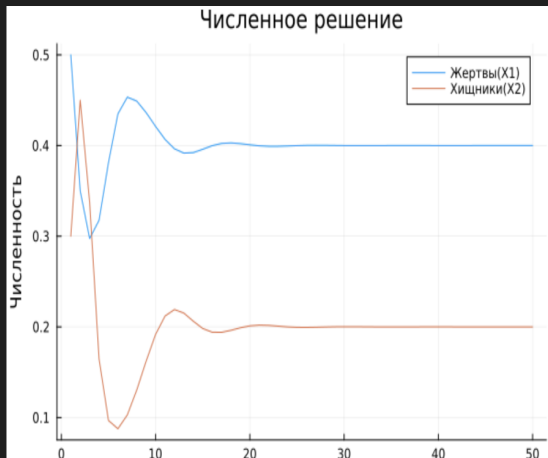
5. Для дискретной модели Лотки-Вольтерры найдите точку равновесия. Получите и сравните аналитическое и численное решения. Численное решение изобразите на фазовом портрете.

```
a = 2
c = 1
d = 5
X0 = [0.5, 0.3]
tspan = 50
X1 = zeros(tspan)
X2 = zeros(tspan)
X1[1] = X0[1]
X2[1] = X0[2]
for t in 1:tspan-1
    X1[t+1] = a * X1[t] * (1-X1[t]) - X1[t] * X2[t]
    X2[t+1] = -c * X2[t] + d * X1[t] * X2[t]
end
plot(1:tspan, X1, label="Жертвы(X1)", xlabel="Время", ylabel="Численность", title="Численное решение")
plot(1:tspan, X2, label="Хищники(X2)")
```

22 Самостоятельное выполнение

```
plot(1:tspan, X1, label="Жертвы(X1)", xlabel="Время", ylabel="Численность", title="Численное решение")  
plot(1:tspan, X2, label="Хищники(X2)")
```

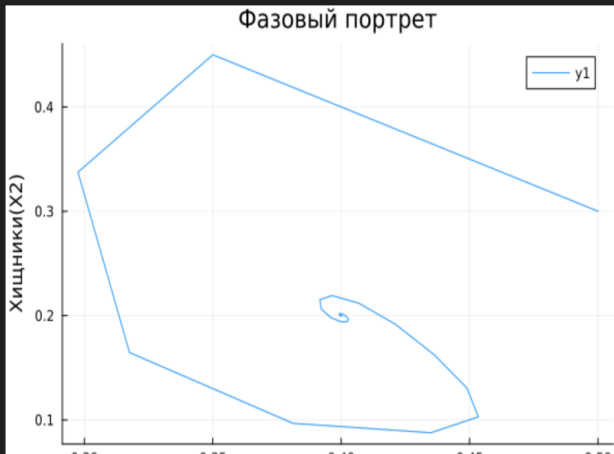
✓ 0.1s



23 Самостоятельное выполнение

```
plot(X1, X2, xlabel="Жертвы(X1)", ylabel="Хищники(X2)", title="Фазовый портрет")
```

✓ 0.0s



24 Самостоятельное выполнение

6. Реализовать на языке Julia модель отбора на основе конкурентных отношений. Начальные данные и параметры задать самостоятельно и пояснить их выбор. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
function competition(du, u, p, t)
    α, β = p
    x, y = u
    du[1] = α * x - β * x * y
    du[2] = α * y - β * x * y
end
```

[36] ✓ 0.0s

Julia

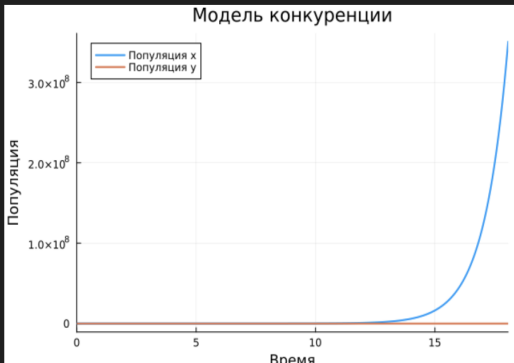
25 Самостоятельное выполнение

```
tspan = (0.0, 50.0)
u0 = [10.0, 5.0]
p = [1, 0.01]
prob = ODEProblem(competition, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, label=["Популяция x" "Популяция y"], title="Модель конкуренции", xlabel="Время", ylabel="Популяция", linewidth=2)
```

[37] ✓ 3.9s

Julia

Warning: Interrupted. Larger maxiters is needed. If you are using an integrator for non-stiff ODEs or an automatic switching algorithm (the default), you
@ SciMLBase [C:\Users\durda\.julia\packages\SciMLBase\TZ9Rx\src\integrator_interface.jl:626](https://github.com/SciML/SciMLBase.jl/blob/master/src/integrator_interface.jl#L626)



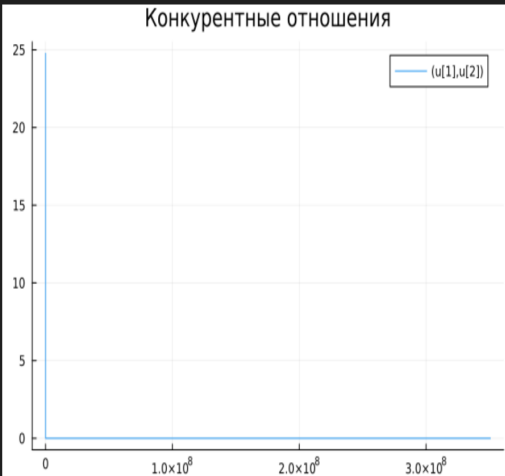
26 Самостоятельное выполнение

```
plot(sol, idxs=(1, 2), title="Конкурентные отношения")
```

[38] ✓ 0.6s

Julia

...



27 Самостоятельное выполнение

7. Реализовать на языке Julia модель консервативного гармонического осциллятора. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.



```
function harmonic(u, p, t)
    x, y = u
    Y, omega = p
    dx = y
    dy = -2*Y*y - omega^2 * x
    return [dx, dy]
end
```

[39] ✓ 0.0s

Julia

... harmonic (generic function with 1 method)

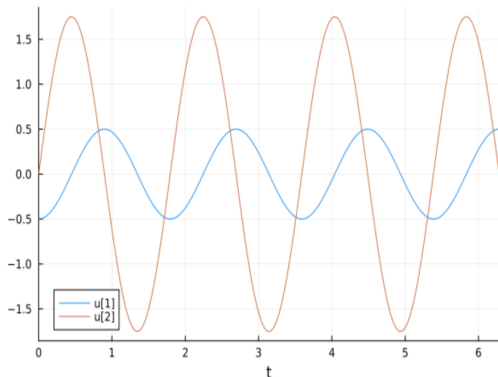
28 Самостоятельное выполнение

```
tspan = (0.0, 2*pi)
u0 = [-0.5, 0]
p = [0, 3.5]
prob = ODEProblem(harmonic, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, title="Гармонический осциллятор")
```

[40] ✓ 1.3s

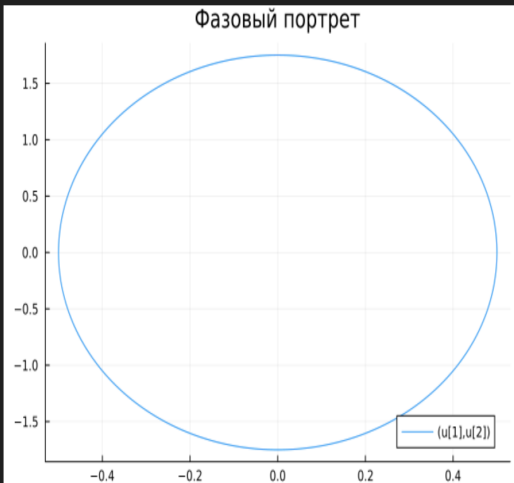
Julia

Гармонический осциллятор



29 Самостоятельное выполнение

```
> ✓ 0.1s  
plot(sol, idxs=(1, 2), title="Фазовый портрет")  
Julia
```



30 Самостоятельное выполнение

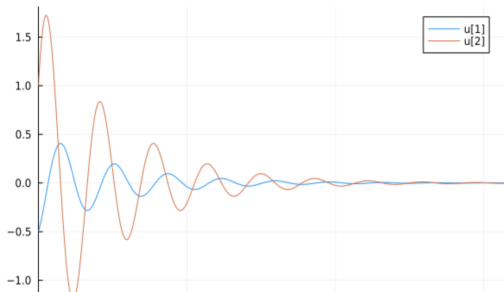
8. Реализовать на языке Julia модель свободных колебаний гармонического осциллятора. Начальные параметры подобрать самостоятельно, выбор пояснить. Построить соответствующие графики (в том числе с анимацией) и фазовый портрет.

```
tspan = (0.0, 5*pi)
u0 = [-0.5, 1]
p = [0.4, 3.5]
prob = ODEProblem(harmonic, u0, tspan, p)
sol = solve(prob, Tsit5())
plot(sol, title="Модель свободных колебаний гармонического осциллятора")
```

✓ 0.0s

Julia

Модель свободных колебаний гармонического осцилля



31 Самостоятельное выполнение



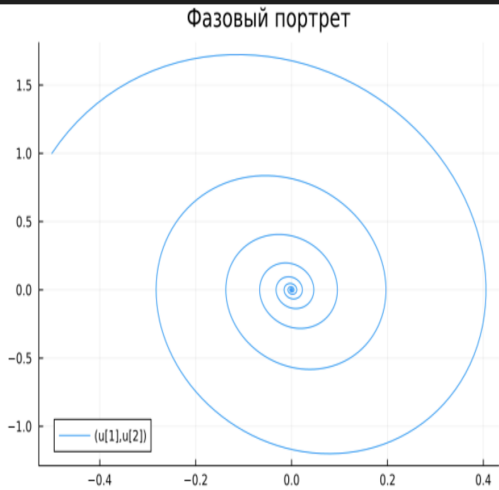
```
plot(sol, vars=(1, 2), title="Фазовый портрет")
```

[43]

✓ 0.0s

Julia

...



В результате выполнения данной лабораторной работы я освоил специализированные пакеты для решения задач в непрерывном и дискретном времени.